

Шпаргалка по Jenkins

Декларативный Jenkinsfile, шаги и условия, параметры и переменные окружения, учётные данные, агенты, артефакты и тесты, обслуживание сервера и типовые неполадки.

Каркас конвейера

```
pipeline {
  agent any
  options {
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
  }
  stages {
    stage('Сборка') {
      steps {
        sh 'make build'
      }
    }
    stage('Тесты') {
      steps {
        sh 'make test'
      }
    }
  }
  post {
    always { junit 'reports/*.xml' }
    failure { echo 'сборка упала' }
  }
}
```

Файл называется Jenkinsfile и лежит в корне репозитория. Декларативный синтаксис строже скриптового, но проще читается и лучше проверяется.

Где что выполняется

agent any
Любой свободный исполнитель

agent { label 'linux && docker' }
Узел с нужными метками

agent { docker { image 'node:20' } }
Шаги выполняются внутри контейнера

agent none
Глобального агента нет: задаётся отдельно на каждом этапе

options { skipDefaultCheckout() }
Отключить автоматическое получение исходников

Частые шаги

sh 'команда' / bat 'команда'
Оболочка Unix / командная строка Windows

checkout scm
Получить код той ветки, для которой запущена сборка

echo "текст"
Сообщение в журнал сборки

dir('subproject') { ... }
Выполнить шаги в подкаталоге

withEnv(['TZ=UTC']) { ... }
Переменные окружения для блока

retry(3) { sh './flaky.sh' }
Повторить при ошибке

timeout(time: 5, unit: 'MINUTES') { ... }
Ограничение времени блока

stash / unstash
Передать файлы между этапами на разных агентах

Параметры и переменные

```
parameters {
  string(name: 'VERSION', defaultValue: '1.0.0')
  booleanParam(name: 'DEPLOY', defaultValue: false)
  choice(name: 'ENV', choices: ['dev', 'stage', 'prod'])
}
```

params.VERSION
Значение параметра внутри конвейера

env.BUILD_NUMBER, env.BRANCH_NAME
Номер сборки и ветка

env.WORKSPACE, env.JOB_NAME
Каталог сборки и имя задания

environment { APP = 'demo' }
Свои переменные для всего конвейера или отдельного этапа

currentBuild.result / displayName
Результат и подпись сборки, доступны для изменения

Учётные данные

```
environment {
  TOKEN = credentials('api-token')
}

steps {
  withCredentials([usernamePassword(
    credentialsId: 'registry',
    usernameVariable: 'USER',
    passwordVariable: 'PASS')]) {
    sh 'echo $PASS | docker login -u $USER --password-stdin'
  }
}
```

Осторожно: секреты маскируются в журнале, но их легко раскрыть неаккуратной командой вроде `set -x` или выводом переменных. Не передавайте их аргументами командной строки.

Условия и ветвление

```
stage('Выкладка') {
  when {
    branch 'master'
    expression { params.DEPLOY }
  }
  steps { sh './deploy.sh' }
}
```

when { not { branch 'master' } }
Отрицание; есть также `anyOf` и `allOf`

when { changeset "src/" }**
Только если менялись указанные файлы

input message: 'Выкладывать?'
Пауза с подтверждением от человека

parallel { stage('A') {...} stage('B') {...} }
Параллельные этапы

catchError(buildResult: 'UNSTABLE') { ... }
Не ронять сборку, но пометить её нестабильной

Результаты сборки

```
junit 'reports/**/*.xml'
    Разбор отчётов тестов: появляется вкладка с
    результатами

archiveArtifacts artifacts: 'build/*.tar.gz',
fingerprint: true
    Сохранить сборочные файлы

publishHTML(...)
    Опубликовать HTML-отчёт, например покрытие

post { success / failure / unstable / always / cleanup
}
    Действия по итогу: уведомления, уборка

cleanWs()
    Очистить рабочий каталог после сборки
```

Организация заданий

- **Multibranch Pipeline** — сам находит ветки и pull request'ы с Jenkinsfile и создаёт для них задания.
- **Shared Library** — общий код конвейеров: подключается через @Library('имя') _.
- **Триггеры**: triggers { cron('H 2 * * *') }, pollSCM или веб-хук из системы контроля версий.
- **Н в расписании** — «любая минута часа»: Jenkins разносит задания во времени, снижая пики нагрузки.

Обслуживание сервера

```
/var/lib/jenkins или $JENKINS_HOME
    Все настройки, задания и ключи; резервная копия —
    это копия каталога

systemctl restart jenkins
    Перезапуск службы

/restart, /safeRestart, /exit
    Адреса управления через веб-интерфейс: немедленно,
    после завершения сборок, остановка

/manage/script
    Консоль Groovy для служебных запросов к серверу

java -jar jenkins-cli.jar -s URL команда
    Управление из командной строки: build, list-jobs, safe-
    restart

Manage Jenkins → Nodes
    Агенты, метки и число исполнителей
```

Когда сборка падает

- Читайте журнал снизу вверх: первая настоящая ошибка часто выше последних строк.
- Ошибка есть только на сервере — сравните версии инструментов на агенте и у себя, а также переменные окружения.
- Кончилось место — df -h на агенте, затем очистка старых сборок через buildDiscarder.
- Задание «висит» — обычно ждёт свободного исполнителя или подтверждения input.
- Правки Jenkinsfile проверяйте в ветке через Multibranch, а синтаксис — линтером: jenkins-cli declarative-linter.